

```
In [10]: # CELL 0
# OBTAINING AND LOADING DATA
import pandas as pd

# Load the datasets
marketing_ads_data = pd.read_csv("marketing_ads_data.csv")
sales_transaction_data = pd.read_csv("sales_transaction_data.csv")

# Preview
print("=== Marketing Ads Data ===")
print(f"shape: {marketing_ads_data.shape}")
print(f"columns: {marketing_ads_data.columns.tolist()}")
print("\n=== Preview 10 baris pertama ===")
print(marketing_ads_data.head(10).to_string(index=False))

print("\n=== Sales Transaction Data ===")
print(f"shape: {sales_transaction_data.shape}")
print(f"columns: {sales_transaction_data.columns.tolist()}")
print("\n=== Preview 10 baris pertama ===")
print(sales_transaction_data.head(10).to_string(index=False))
```

```
=== Marketing Ads Data ===
shape: (1030, 12)
columns: ['ad_id', 'campaign_name', 'audience_type', 'product_type', 'product_name', 'variant', 'date', 'impressions', 'clicks', 'spend_idr', 'sessions', 'add_to_cart']

=== Preview 10 baris pertama ===
   ad_id  campaign_name audience_type product_type product_name variant date impressions
clicks spend_idr sessions add_to_cart
AD_0032  Awareness Campaign      cold      Serum  DermaGlow Brightening Serum  Video_Review  2023-01-06      8060
253  458896.0      216      29
AD_0110  Awareness Campaign      cold  Sunscreen DermaGlow UV Shield Sunscreen  Video_Review  01/09/2023      17314
542  108070.0      452      72
AD_0137  Retargeting Campaign      warm  Sunscreen DermaGlow UV Shield Sunscreen  Static_Catalog  2023-04-13      16374
180  135435.0      146      8
AD_0089  Retargeting Campaign      warm  Sunscreen DermaGlow UV Shield Sunscreen  Static_Catalog  Nov 14, 2023      10735
159  244118.0      128      6
AD_0919  Flash Sale Campaign      cold  Cleanser  DermaGlow Gentle Cleanser  Video_Review  Apr 15, 2023      12446
268      NaN      217      36
AD_0169  Flash Sale Campaign      cold      Serum  DermaGlow Brightening Serum  Video_Review  2023-03-29      13731
443  269631.0      355      43
AD_0871  Retargeting Campaign      warm      Serum  DermaGlow Brightening Serum  Video_Review  Mar 16, 2023      8834
223  236016.0      178      18
AD_0319  Awareness Campaign      cold      Serum  DermaGlow Brightening Serum  Video_Review  2023-12-09      14569
311  287954.0      254      32
AD_0262  Retargeting Campaign      warm      Serum  DermaGlow Brightening Serum  Static_Catalog  2023-08-08      8974
133  248087.0      119      12
AD_0536  Retargeting Campaign      warm      Serum  DermaGlow Brightening Serum  Static_Catalog  23/12/2023      8981
127  326453.0      103      16

=== Sales Transaction Data ===
shape: (2500, 8)
columns: ['transaction_id', 'ad_id', 'transaction_date', 'product_type', 'product_name', 'variant', 'revenue_idr', 'order_status']

=== Preview 10 baris pertama ===
transaction_id  ad_id transaction_date product_type product_name variant revenue_idr order_status
TXN_001448      NaN      2023-03-15  Cleanser  DermaGlow Gentle Cleanser  Static_Catalog  403000.0  Success
TXN_001115 AD_0958      2023-11-30  Moisturizer DermaGlow Hydrating Moisturizer  Video_Review  187000.0  Success
TXN_001065 AD_0145      2023-12-15  Cleanser  DermaGlow Gentle Cleanser  Static_Catalog  0.0  Cancelled
TXN_002288 AD_0349      2023-11-08  Cleanser  DermaGlow Gentle Cleanser  Static_Catalog -187000.0  Refunding
TXN_001538 AD_0317      2023-06-22  Moisturizer DermaGlow Hydrating Moisturizer  Video_Review  236000.0  Success
TXN_000669 AD_0992      2023-11-15      Serum  DermaGlow Brightening Serum  Video_Review  521000.0  Success
TXN_001584 AD_0815      2023-10-12  Sunscreen DermaGlow UV Shield Sunscreen  Video_Review  215000.0  Success
TXN_002405 AD_0698      2023-07-19  Cleanser  DermaGlow Gentle Cleanser  Static_Catalog  264000.0  Success
TXN_000498      NaN      2023-05-26  Cleanser  DermaGlow Gentle Cleanser  Video_Review  0.0  Cancelled
TXN_002481 AD_0543      2023-09-06  Moisturizer DermaGlow Hydrating Moisturizer  Static_Catalog  132000.0  Success
```

```
In [2]: # CELL 1
# SCRUBBING AND PREPARING DATA

print("===== MARKETING ADS DATA =====")

# Data Structure
print("\nStruktur Data:")
print(marketing_ads_data.info())

# Basic Statistics
print("\nStatistik Dasar:")
print(marketing_ads_data.describe())

# Missing Values Check
print("\nMissing Values:")
print(marketing_ads_data.isnull().sum())

# Duplicate Check
```

```
print("\nDuplikat:")
print(marketing_ads_data.duplicated().sum())

# Unique Values Check for Important Columns
print("\n--- Unique Values Check for Important Columns:---")
print("Variant:", marketing_ads_data['variant'].unique())
print("Campaign Name:", marketing_ads_data['campaign_name'].unique())
print("Audience Type:", marketing_ads_data['audience_type'].unique())
print("Product Type:", marketing_ads_data['product_type'].unique())
print("Product Name:", marketing_ads_data['product_name'].unique())

# Data Sample
print("Data Sample:", marketing_ads_data.sample(10))

print("\n==== SALES TRANSACTION DATA =====")

# Data Structure
print("\nStruktur Data:")
print(sales_transaction_data.info())

# Basic Statistics
print("\nStatistik Dasar:")
print(sales_transaction_data.describe())

# Missing Values Check
print("\nMissing Values:")
print(sales_transaction_data.isnull().sum())

# Duplicate Check
print("\nDuplikat:")
print(sales_transaction_data.duplicated().sum())

# Unique Values Check for Important Columns
print("\n--- Unique Values Check for Important Columns:---")
print("Variant:", sales_transaction_data['variant'].unique())
print("Product Type:", sales_transaction_data['product_type'].unique())
print("Product Name:", sales_transaction_data['product_name'].unique())
print("Order Status:", sales_transaction_data['order_status'].unique())

# Data Sample
print("Data Sample:", sales_transaction_data.sample(10))
```

===== MARKETING ADS DATA =====

Struktur Data:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1030 entries, 0 to 1029
Data columns (total 12 columns):
Column Non-Null Count Dtype
--- ----- -----
0 ad_id 1030 non-null object
1 campaign_name 1030 non-null object
2 audience_type 1030 non-null object
3 product_type 1030 non-null object
4 product_name 1030 non-null object
5 variant 1030 non-null object
6 date 1030 non-null object
7 impressions 1030 non-null int64
8 clicks 1030 non-null int64
9 spend_idr 970 non-null float64
10 sessions 1030 non-null int64
11 add_to_cart 1030 non-null int64
dtypes: float64(1), int64(4), object(7)
memory usage: 96.7+ KB
None

Statistik Dasar:

	impressions	clicks	spend_idr	sessions	add_to_cart
count	1030.000000	1030.000000	9.700000e+02	1030.000000	1030.000000
mean	12302.346602	228.747573	3.095047e+05	194.000971	23.620388
std	3796.802668	122.200596	1.934729e+05	104.467101	16.449072
min	5078.000000	47.000000	5.048000e+03	38.000000	3.000000
25%	9206.250000	135.000000	1.975175e+05	115.000000	12.000000
50%	12013.000000	200.500000	3.002635e+05	171.000000	19.000000
75%	15052.250000	296.500000	4.045185e+05	250.750000	31.000000
max	19940.000000	624.000000	2.839487e+06	536.000000	99.000000

Missing Values:
ad_id 0
campaign_name 0
audience_type 0
product_type 0
product_name 0
variant 0
date 0
impressions 0
clicks 0
spend_idr 60
sessions 0
add_to_cart 0
dtype: int64

Duplikat:
30

--- Unique Values Check for Important Columns:---
Variant: ['Video_Review' 'Static_Catalog']
Campaign Name: ['Awareness Campaign' 'Retargeting Campaign' 'Flash Sale Campaign']
Audience Type: ['cold' 'warm']
Product Type: ['Serum' 'Sunscreen' 'Cleanser' 'Moisturizer']
Product Name: ['DermaGlow Brightening Serum' 'DermaGlow UV Shield Sunscreen'
 'DermaGlow Gentle Cleanser' 'DermaGlow Hydrating Moisturizer']
Data Sample: ad_id campaign_name audience_type product_type \
414 AD_0007 Retargeting Campaign warm Serum
597 AD_0150 Flash Sale Campaign cold Serum
198 AD_0990 Flash Sale Campaign cold Serum
988 AD_0567 Awareness Campaign cold Sunscreen
559 AD_0435 Awareness Campaign cold Sunscreen
579 AD_0705 Flash Sale Campaign cold Sunscreen
737 AD_0148 Awareness Campaign cold Sunscreen
37 AD_0970 Flash Sale Campaign cold Sunscreen
852 AD_0564 Flash Sale Campaign cold Sunscreen
122 AD_0353 Retargeting Campaign warm Cleanser

	product_name	variant	date	impressions	\
414	DermaGlow Brightening Serum	Static_Catalog	2023-11-06	13873	
597	DermaGlow Brightening Serum	Static_Catalog	08/09/2023	16105	
198	DermaGlow Brightening Serum	Video_Review	2023-05-16	14192	
988	DermaGlow UV Shield Sunscreen	Video_Review	2023-03-26	10982	
559	DermaGlow UV Shield Sunscreen	Video_Review	2023-04-02	13411	
579	DermaGlow UV Shield Sunscreen	Static_Catalog	2023-06-15	14630	
737	DermaGlow UV Shield Sunscreen	Video_Review	Nov 25, 2023	6380	
37	DermaGlow UV Shield Sunscreen	Video_Review	2023-10-12	11284	
852	DermaGlow UV Shield Sunscreen	Video_Review	2023-09-03	14974	
122	DermaGlow Gentle Cleanser	Video_Review	2023-06-30	9222	

	clicks	spend_idr	sessions	add_to_cart
414	161	137065.0	143	19
597	144	488411.0	116	21

198	404	142678.0	350	26
988	281	NaN	230	33
559	313	208944.0	258	36
579	163	442016.0	142	8
737	120	183560.0	100	9
37	351	372217.0	283	19
852	312	255918.0	268	14
122	273	486721.0	223	25

===== SALES TRANSACTION DATA =====

Struktur Data:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2500 entries, 0 to 2499
Data columns (total 8 columns):
Column Non-Null Count Dtype
--- ---
0 transaction_id 2500 non-null object
1 ad_id 2408 non-null object
2 transaction_date 2500 non-null object
3 product_type 2500 non-null object
4 product_name 2500 non-null object
5 variant 2500 non-null object
6 revenue_idr 2500 non-null float64
7 order_status 2500 non-null object
dtypes: float64(1), object(7)
memory usage: 156.4+ KB
None

Statistik Dasar:
 revenue_idr
count 2.500000e+03
mean 9.273054e+05
std 1.650125e+07
min -4.450000e+05
25% 8.675000e+04
50% 2.175000e+05
75% 3.580000e+05
max 4.692353e+08

Missing Values:
transaction_id 0
ad_id 92
transaction_date 0
product_type 0
product_name 0
variant 0
revenue_idr 0
order_status 0
dtype: int64

Duplikat:
0

--- Unique Values Check for Important Columns:---
Variant: ['Static_Catalog' 'Video_Review']
Product Type: ['Cleanser' 'Moisturizer' 'Serum' 'Sunscreen']
Product Name: ['DermaGlow Gentle Cleanser' 'DermaGlow Hydrating Moisturizer'
 'DermaGlow Brightening Serum' 'DermaGlow UV Shield Sunscreen']
Order Status: ['Success' 'Cancelled' 'Refunding']
Data Sample: transaction_id ad_id transaction_date product_type \
51 TXN_001756 AD_0440 2023-09-28 Serum
160 TXN_001365 AD_0429 2023-05-19 Sunscreen
2063 TXN_001904 AD_0084 2023-12-29 Sunscreen
2363 TXN_001664 AD_0602 2023-06-23 Moisturizer
937 TXN_002226 AD_0359 2023-12-26 Moisturizer
933 TXN_001906 AD_0084 2023-12-29 Sunscreen
720 TXN_000314 AD_0469 2023-02-09 Serum
2240 TXN_001046 NaN 2023-03-17 Moisturizer
689 TXN_002366 AD_0361 2023-02-26 Sunscreen
1772 TXN_002101 AD_0812 2023-08-12 Sunscreen

	product_name	variant	revenue_idr \
51	DermaGlow Brightening Serum	Static_Catalog	230000.0
160	DermaGlow UV Shield Sunscreen	Video_Review	306000.0
2063	DermaGlow UV Shield Sunscreen	Static_Catalog	-122000.0
2363	DermaGlow Hydrating Moisturizer	Video_Review	304000.0
937	DermaGlow Hydrating Moisturizer	Video_Review	229000.0
933	DermaGlow UV Shield Sunscreen	Static_Catalog	350000.0
720	DermaGlow Brightening Serum	Static_Catalog	0.0
2240	DermaGlow Hydrating Moisturizer	Static_Catalog	0.0
689	DermaGlow UV Shield Sunscreen	Static_Catalog	0.0
1772	DermaGlow UV Shield Sunscreen	Static_Catalog	343000.0

	order_status
51	Success
160	Success

2063	Refunding
2363	Success
937	Success
933	Success
720	Cancelled
2240	Cancelled
689	Cancelled
1772	Success

FINDINGS:

- **Marketing Ads 3 masalah**
 - Kolom Spend IDR = 60 nulls (5.8% dari data)
 - Kolom Date format masih object, belum datetime
 - 30 duplikat
- **Sales Transaction 3 masalah**
 - Kolom Date format masih object, belum datetime
 - Kolom Revenue IDR ada nilai negatif (min: -445.000) dan nilai (max: 469 juta) kemungkinan outlier.
 - Kolom ad_id 92 null transaksi tanpa tracking
 - Kolom Revenue IDR ada nilai 0.0, kemungkinan status order cancelled, perlu konfirmasi.

```
In [3]: # CELL 2
# DATA CLEANING & TRANSFORMATION

# MARKETING ADS DATA

# 1. Drop duplicates
marketing_ads_data = marketing_ads_data.drop_duplicates()
print("Marketing Ads setelah drop duplicates:", marketing_ads_data.shape)

# 2. Standarisasi kolom Date
marketing_ads_data['date'] = pd.to_datetime(
    marketing_ads_data['date'], format='mixed', dayfirst=True
)
print("Tipe data kolom date:", marketing_ads_data['date'].dtype)

# 3. Investigasi pola null sebelum treatment
print("\n === Null spend_idr per campaign_name: ===")
print(marketing_ads_data[marketing_ads_data['spend_idr'].isnull()][ 'campaign_name'].value_counts())

print("\n === Null spend_idr per audience_type: ===")
print(marketing_ads_data[marketing_ads_data['spend_idr'].isnull()][ 'audience_type'].value_counts())

# SALES TRANSACTIONS DATA

# 1. Standarisasi kolom Date
sales_transaction_data['date'] = pd.to_datetime(sales_transaction_data['transaction_date'], format='mixed', dayfirst=True)
print("Tipe data kolom date:", sales_transaction_data['date'].dtype)

# 2. investigasi kolom revenue_idr negatif
print("\n === revenue_idr negatif per order_status ===")
print(sales_transaction_data[sales_transaction_data['revenue_idr'] < 0][ 'order_status'].value_counts())
print("Jumlah baris revenue negatif:", sales_transaction_data[sales_transaction_data['revenue_idr'] < 0].shape[0])

# 3. Investigasi revenue outlier ekstrim (> 10 juta)
print("\n === Revenue ekstrim (>10 juta) ===")
print(sales_transaction_data[sales_transaction_data['revenue_idr'] > 10_000_000][ 'transaction_id', 'revenue_idr', 'order_stat

# Drop kolom date karena sudah ada kolom transaction_date yang Lebih spesifik
sales_transaction_data = sales_transaction_data.drop(columns=['date'])
print("\nSales Transactions columns setelah drop date:")
print(sales_transaction_data.columns.tolist())
```

Marketing Ads setelah drop duplicates: (1000, 12)
Tipe data kolom date: datetime64[ns]

```
=== Null spend_idr per campaign_name: ===
campaign_name
Flash Sale Campaign      26
Awareness Campaign       17
Retargeting Campaign     13
Name: count, dtype: int64
```

```
=== Null spend_idr per audience_type: ===
audience_type
cold      43
warm      13
Name: count, dtype: int64
Tipe data kolom date: datetime64[ns]
```

```
=== revenue_idr negatif per order_status ===
order_status
Refunding      209
Name: count, dtype: int64
Jumlah baris revenue negatif: 209
```

```
=== Revenue ekstrim (>10 juta) ===
transaction_id  revenue_idr  order_status
708      TXN_001591  266506890.0      Success
945      TXN_001160  348177883.0      Success
1393     TXN_001980  299801663.0      Success
2227     TXN_000547  424370733.0      Success
2410     TXN_000419  469235332.0      Success
```

Sales Transactions columns setelah drop date:
['transaction_id', 'ad_id', 'transaction_date', 'product_type', 'product_name', 'variant', 'revenue_idr', 'order_status']

CELL 2 - DATA CLEANING OUTPUT

MARKETING ADS - kolom spend_idr 56 nulls:

Null tersebar di semua campaign (Flash Sale: 26, Awareness Campaign: 17, Retargeting Campaign: 13), mayoritas pada Cold audience (43) kemungkinan missing at random. Decision: fill null dengan median per campaign_name (lebih robust dari mean karna ada outlier di spend).

SALES TRANSACTION DATA - 3 Decisions

- revenue_idr negatif semua dari Refunding (209) valid secara bisnis, tidak saya drop tapi akan saya konversi ke absolut.
- 5 baris revenue ekstrim (266 juta - 469 juta) dengan status success, asumsi merupakan data anomali dan bukan transaksi normal produk skincare, decision: drop.
- kolom ad_id null 92 baris tidak di drop karena transaksi tetap valid untuk analisis revenue.

```
In [4]: # CELL 3
# DATA TREATMENT

# MARKETING ADS DATA

# Fill null spend_idr dengan median per campaign_name
marketing_ads_data['spend_idr'] = marketing_ads_data.groupby('campaign_name')['spend_idr'].transform(lambda x: x.fillna(x.median()))
print("Null spend_idr setelah fill:", marketing_ads_data['spend_idr'].isnull().sum())

# SALES TRANSACTIONS DATA

# 1. Drop revenue ekstrim
before = sales_transaction_data.shape[0]
sales_transaction_data = sales_transaction_data[sales_transaction_data['revenue_idr'] <= 10_000_000]
print(f"Baris di drop karna revenue ekstrim: {before - sales_transaction_data.shape[0]}")

# 2. Revenue negatif Refunding di konversi ke absolut
sales_transaction_data['revenue_idr'] = sales_transaction_data['revenue_idr'].abs()
print("Min revenue_idr setelah konversi:", sales_transaction_data['revenue_idr'].min())

# FINAL CHECK

print("\n=== Final Shape ===")
print("Marketing Ads:", marketing_ads_data.shape)
print("Sales Transactions:", sales_transaction_data.shape)

print("\n=== Final Null Check ===")
print(marketing_ads_data.isnull().sum())
print(sales_transaction_data.isnull().sum())
```


Null spend_idr setelah fill: 0
Baris di drop karna revenue ekstrim: 5
Min revenue_idr setelah konversi: 0.0

=== Final Shape ===
Marketing Ads: (1000, 12)
Sales Transactions: (2495, 8)

=== Final Null Check ===
ad_id 0
campaign_name 0
audience_type 0
product_type 0
product_name 0
variant 0
date 0
impressions 0
clicks 0
spend_idr 0
sessions 0
add_to_cart 0
dtype: int64
transaction_id 0
ad_id 92
transaction_date 0
product_type 0
product_name 0
variant 0
revenue_idr 0
order_status 0
dtype: int64

CELL 3

SUMMARY

- 1. kolom spend_idr null diisi dengan nilai median per campaign tidak ada lagi kolom null tersisa.
- 2. revenue ekstrim sudah tidak ada.
- 3. revenue negatif di konversi ke absolut.
- 4. Shape Final: Marketing Ads: (1000, 12), Sales Transactions: (2495, 8).

```
In [5]: # CELL 4
# MERGE DATA

# 1. Agregasi revenue per ad_id dari sales transaction. Hanya ambil transaksi Success saja untuk revenue yang valid
revenue_per_ad = sales_transaction_data[sales_transaction_data['order_status'] == 'Success'].groupby('ad_id').agg(
    total_revenue=('revenue_idr', 'sum'),
    total_orders=('transaction_id', 'count')
).reset_index()

print("Shape revenue_per_ad:", revenue_per_ad.shape)
print(revenue_per_ad.head())

# 2. Merge dengan marketing ads data
merged_data = marketing_ads_data.merge(revenue_per_ad, on='ad_id', how='left')

print("\nShape merged_data:", merged_data.shape)
print(merged_data.isnull().sum())

# 3. Fill null total_revenue dan total_orders dengan 0
merged_data[['total_revenue', 'total_orders']] = merged_data[['total_revenue', 'total_orders']].fillna(0)

print("Null setelah fill:")
print(merged_data[['total_revenue', 'total_orders']].isnull().sum())
print("Shape:", merged_data.shape)
```



```
Shape revenue_per_ad: (587, 3)
  ad_id  total_revenue  total_orders
0  AD_0001      1450000.0           5
1  AD_0002       723000.0           2
2  AD_0004       806000.0           3
3  AD_0006       636000.0           2
4  AD_0007      1045000.0           3
```

```
Shape merged_data: (1000, 14)
ad_id      0
campaign_name  0
audience_type  0
product_type  0
product_name  0
variant      0
date         0
impressions  0
clicks       0
spend_idr    0
sessions     0
add_to_cart  0
total_revenue 413
total_orders 413
dtype: int64
Null setelah fill:
total_revenue  0
total_orders   0
dtype: int64
Shape: (1000, 14)
```

CELL 4 (MERGE DATA) - INSIGHT & KEY FINDINGS

Agregasi dilakukan sebelum Merge Data karena pada kasus ini, kolom 'ad_id' di data Sales Transactions bisa banyak transaksi per Ad. Revenue perlu agregasi per Ad di Sales Transaction sebelum merge agar tidak terjadi row explosion.

Yang Saya Lakukan:

Melakukan agregasi sebelum merge data. Join nya adalah Marketing Ads Data LEFT JOIN Sales Transaction Data on ad_id agar semua data Ad tetap terhitung meski tanpa transaksi.

```
In [6]: # CELL 5
# KALKULASI METRIK

merged_data['CTR'] = merged_data['clicks'] / merged_data['impressions']
merged_data['CPC'] = merged_data['spend_idr'] / merged_data['clicks']
merged_data['ROAS'] = merged_data['total_revenue'] / merged_data['spend_idr']
merged_data['CVR'] = merged_data['total_orders'] / merged_data['sessions']

print("=== Sample Metrics ===")
print(merged_data[['ad_id', 'CTR', 'CPC', 'ROAS', 'CVR']].head(10))

print("\n=== Statistic Metric per Variant ===")
print(merged_data.groupby('variant')[['CTR', 'CPC', 'ROAS', 'CVR']].mean().round(4))

=== Sample Metrics ===
   ad_id  CTR      CPC      ROAS      CVR
0  AD_0032 0.031390 1813.818182  0.000000 0.000000
1  AD_0110 0.031304  199.391144  0.000000 0.000000
2  AD_0137 0.010993  752.416667 17.240743 0.054795
3  AD_0089 0.014811 1535.333333  7.144086 0.054688
4  AD_0919 0.021533 1147.164179  1.766198 0.004608
5  AD_0169 0.032263  608.647856  5.600246 0.011268
6  AD_0871 0.025243 1058.367713  3.779405 0.011236
7  AD_0319 0.021347  925.897106  0.000000 0.000000
8  AD_0262 0.014821 1865.315789  5.111110 0.042017
9  AD_0536 0.014141 2570.496063  0.000000 0.000000

=== Statistic Metric per Variant ===
              CTR      CPC      ROAS      CVR
variant
Static_Catalog 0.0119 2538.6264  3.4047  0.0191
Video_Review   0.0250 1117.3851  1.7696  0.0051

              CTR      CPC      ROAS      CVR
variant
Static_Catalog 0.0119 2538.6264  3.4047  0.0191
Video_Review   0.0250 1117.3851  1.7696  0.0051
```

CELL 5 (KALKULASI METRIK) - INSIGHT & KEY FINDINGS

Yang Saya Lakukan:

Melakukan kalkulasi untuk metrik yang dibutuhkan seperti CPC, CVR, ROAS dan CTR.

Key Findings:

- **Finding 1:** CTR: Video Review unggul (2% vs 1%), orang lebih banyak klik video daripada catalog
- **Finding 2:** CPC: Video Review lebih efisien secara cost per click (Rp. 1,117 vs Rp. 2,538).
- **Finding 3:** ROAS: Static Catalog unggul (3% vs 2%), indikasi audience pada Catalog memiliki purchase intent yang tinggi.
- **Finding 4:** CVR: Kedua variant hampir sama persis (Static catalog 0.8459 vs Video Review 0.8481)

Perbedaan ini akan diuji signifikansinya pada CELL selanjutnya

```
In [7]: # CELL 6 - EXPLORATORY DATA ANALYSIS (EDA)
# STATISTICAL SIGINIFCANCE TESTING (SHAPIRO WILK TEST & MANN WHITNEY U TEST)

from scipy import stats

# 1. Data per Variant dipisahkan dahulu
video = merged_data[merged_data['variant'] == 'Video_Review']
static = merged_data[merged_data['variant'] == 'Static_Catalog']

# 2. Cek normalitas menggunakan Shapiro Wilk Test
print("=== Shapiro Wilk Test for Normality ===")
for metric in ['CTR', 'CVR', 'CPC', 'ROAS']:
    _, p_video = stats.shapiro(video[metric])
    _, p_static = stats.shapiro(static[metric])
    print(f"{metric} - Video p: {p_video:.4f} | Static p: {p_static:.4f}")

# 3. Cek signifikansi perbedaan antara kedua variant menggunakan Mann Whitney U Test (karena data tidak normal)
print("\n=== Mann-Whitney U Test ===")
for metric in ['CTR', 'CVR', 'CPC', 'ROAS']:
    stat, p_value = stats.mannwhitneyu(
        video[metric], static[metric], alternative='two-sided'
    )
    significance = "SIGNIFICANT" if p_value < 0.05 else "not significant"
    print(f"{metric} - p-value: {p_value:.4f} | {significance}")

=== Shapiro Wilk Test for Normality ===
CTR - Video p: 0.0000 | Static p: 0.0000
CVR - Video p: 0.0000 | Static p: 0.0000
CPC - Video p: 0.0000 | Static p: 0.0000
ROAS - Video p: 0.0000 | Static p: 0.0000

=== Mann-Whitney U Test ===
CTR - p-value: 0.0000 | SIGNIFICANT
CVR - p-value: 0.0000 | SIGNIFICANT
CPC - p-value: 0.0000 | SIGNIFICANT
ROAS - p-value: 0.0001 | SIGNIFICANT
```

CELL 6 MARKDOWN

Findings:

- Shapiro-Wilk menunjukkan semua metrik tidak berdistribusi normal, Mann-Whitney U dipilih sebagai non parametric alternatif
- Semua metrik (CTR, CVR, CPC, ROAS) menunjukkan perbedaan yang signifikan secara statistik antara Video_Review dan Static_Catalog (p < 0.05).

```
In [8]: # CELL 7 - EXPLORATORY DATA ANALYSIS (EDA)
# SEGMENT ANALYSIS

# 1. Per Audience Type
print("=== Segment Analysis: Audience Type ===")
print(merged_data.groupby(['audience_type', 'variant'])[['CTR', 'CVR', 'CPC', 'ROAS']].mean().round(4))

# 2. Per Product Type
print("\n=== Segment Analysis: Product Type ===")
print(merged_data.groupby(['product_type', 'variant'])[['CTR', 'CVR', 'CPC', 'ROAS']].mean().round(4))
```

=== Segment Analysis: Audience Type ===					
		CTR	CVR	CPC	ROAS
audience_type	variant				
cold	Static_Catalog	0.0120	0.0180	2568.7309	2.9863
	Video_Review	0.0250	0.0050	1145.0309	1.7150
warm	Static_Catalog	0.0117	0.0213	2479.4798	4.2268
	Video_Review	0.0252	0.0053	1058.7901	1.8855
=== Segment Analysis: Product Type ===					
		CTR	CVR	CPC	ROAS
product_type	variant				
Cleanser	Static_Catalog	0.0119	0.0196	2574.7858	2.2872
	Video_Review	0.0247	0.0065	1011.8212	2.1635
Moisturizer	Static_Catalog	0.0116	0.0181	2527.2755	3.0546
	Video_Review	0.0248	0.0042	1215.6686	1.3945
Serum	Static_Catalog	0.0120	0.0187	2597.6584	3.6989
	Video_Review	0.0251	0.0049	1097.4098	2.0389
Sunscreen	Static_Catalog	0.0121	0.0201	2463.4566	4.3527
	Video_Review	0.0256	0.0051	1111.5418	1.6006

CELL 7 - MARKDOWN

INSIGHT AND FINDINGS

Findings:

- **Finding 1:** Video_Review selalu unggul di CTR dan CPC pada semua segment
- **Finding 2:** Static_Catalog selalu unggul di CVR dan ROAS
- **Finding 3:** Warm audience static catalog tertinggi (4.23), paling tinggi di semua segment
- **Finding 4:** Sunscreen static catalog ROAS tertinggi (4.35), produk return paling tinggi
- **Finding 5:** Moisturizer video review ROAS buruk di Video Review (1.40).

Insight:

Video Review efisien pada top funnel sedangkan Static Catalog efektif pada bottom funnel, indikasi Video berfungsi untuk mendapatkan attention dan kemudian conversion melalui Static Catalog. Sunscreen menjadi prioritas utama campaign karena menghasilkan return tertinggi dalam segment product type.

CELL 8 - BUSINESS QUESTION & RECCOMMENDATION

Business Question 1: Variant mana yang perform lebih baik overall?

Tidak ada 1 variant yang unggul di semua metrik, kedua variant unggul di area yang berbeda - Video Review: Unggul di Top Funnel: CTR (0.0250 vs 0.0119), CPC (Rp. 1,117 vs Rp. 2,538). - Static Catalog: Unggul di Bottom Funnel: CVR (0.8459 vs 0.8481), ROAS (3.49 vs 1.77).

Business Question 2: Apakah perbedaan performa tersebut signifikan secara statistik?

Berdasarkan hasil testing dengan MANN WHITNEY U TEST menunjukkan perbedaan secara signifikan (p < 0.05) di semua metrik. **Metrik:** - CTR : < 0.0001 Signifikan - CVR : < 0.0001 Signifikan - CPC : < 0.0001 Signifikan - ROAS : < 0.0001 Signifikan Perbedaan performa bukan kebetulan, valid menjadi dasar keputusan bisnis.

Business Question 3: Bagaimana performa masing - masing variant di berbagai segment?

1. Audience Type

- Warm Audience konsisten menghasilkan ROAS lebih tinggi di kedua variant, Static Catalog Warm Audience menacapai ROAS 4.23 tertinggi dari semua segment audience
- Cold Audience lebih responsif terhadap Video Review dari sisi CTR namun ROAS tetap lebih rendah dari Static Catalog.

2. Product Type

- Sunscreen adalah produk dengan ROAS tertinggi di variant Static Catalog (4.35)
- Moisturizer adalah kombinasi terlemah pada variant Video Review, ROAS (1.35)
- Static Catalog konsisten unggul ROAS di semua product type.

Business Question 4: Apakah ada trade off antara CTR dan Conversion Rate?

Trade Off konsisten di semua segment. - Video Review menarik lebih banyak klik (CTR tinggi) namun menghasilkan revenue per spend yang lebih rendah (ROAS rendah) - Static Catalog menghasilkan lebih sedikit klik namun setiap klik yang dihasilkan berkualitas, dibuktikan dengan ROAS yang lebih tinggi - Ini mengindikasikan bahwa audience yang tertarik lewat variant Static Catalog memiliki purchase intent lebih tinggi

Business Question 5: Dari segi bisnis, mana lebih layak di scale?

Rekomendasi hybrid strategy.

Tidak disarankan memilih salah satu variant secara eksklusif.

1. Scale Static Catalog untuk:
- Warm Audience, ROAS 4.23 purchase intent tinggi
 - Produk Sunscreen dan Serum, ROAS tertinggi di produk ini
 - Campaign yang berfokus pada revenue dan profit
2. Scale Video Review untuk:
- Cold Audience, efektif untuk menarik audience baru dengan CPC lebih efisien
 - Top of Funnel campaign untuk reach dan engagement bukan immediate conversion.

Next Step yang disarankan

1. ALokasikan budget lebih besar pada Static Catalog untuk warm audience
2. Optimalkan landing page untuk Video Review, CVR nya rendah padahaln traffic tinggi, kemungkinan ada friction di post click experience
3. Lakukan AB test lanjutan untuk moisturizer, kombinasi Video Review + Moisturizer paling underperform dan perlu investigasi lebih lanjut.

In [9]:

```
# CELL 9 - EXPORT DATASET

merged_data.to_csv("dermaglow_merged_data_cleaned.csv", index=False)
print("Data Exported:", merged_data.shape)
```

Data Exported: (1000, 18)
(1000, 18)